BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 3

Roll no: 37 PRN No: 12412175 Name of Student: Rugved Raut

Subject Name : Deep Learning

Problem Statement

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm

1. Start.
2. Load the Tomato Disease dataset.
3. Identify the different disease classes in the dataset.
4. Resize all images to 128 × 128 pixels.
5. Normalize the image pixel values by dividing them by 255.
6. Split the dataset into training and validation datasets.
7. Create a CNN model.
8. Add a convolutional layer with 32 filters of size 3×3 and apply ReLU activation.
9. Add a 2×2 Max Pooling layer.
10. Add a convolutional layer with 64 filters of size 3×3 and apply ReLU activation.
11. Add another 2×2 Max Pooling layer.
12. Add a convolutional layer with 128 filters of size 3×3 and apply ReLU activation.
13. Add another 2×2 Max Pooling layer.
14. Flatten the extracted features into a one-dimensional vector.
15. Add a Dense layer with 128 neurons and apply ReLU activation.
16. Add a Dropout layer with a rate of 0.5 to reduce overfitting.
17. Add the output layer with the number of neurons equal to the number of disease classes and apply Softmax activation.
18. Compile the CNN using the Adam optimizer, categorical cross-entropy loss, and accuracy as the evaluation metric.
19. Train the CNN using the training dataset for the specified number of epochs.
20. Validate the model using the validation dataset.
21. Evaluate the model using accuracy, loss, precision, recall, F1-score, and a confusion matrix.
22. Upload a new tomato leaf image for testing.
23. Preprocess the test image by resizing and normalizing it.
24. Pass the image through the trained CNN model.
25. Predict the disease class with the highest probability.

26. Display the predicted disease and confidence score.
27. Stop.

Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import os

from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
print("TensorFlow Version:", tf.__version__)
print("GPU:", tf.config.list_physical_devices('GPU'))
from google.colab import files
uploaded = files.upload()
import os
os.makedirs('/root/.kaggle', exist_ok=True)
!cp kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
!kaggle datasets download -d emmarex/plantdisease
!unzip -q plantdisease.zip -d tomato_dataset
!ls tomato_dataset
import os
for root, dirs, files in os.walk("tomato_dataset"):
    print(root)
    if len(files) > 0:
        print("Number of images:", len(files))
    if root.count(os.sep) > 3:
        dirs[:] = []
dataset_path = "tomato_dataset/PlantVillage"
print(os.listdir(dataset_path))
IMG_SIZE = 128
```

```
BATCH_SIZE = 32
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)
train_data = datagen.flow_from_directory(
    dataset_path,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training',
    shuffle=True
)
validation_data = datagen.flow_from_directory(
    dataset_path,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation',
    shuffle=False
)
class_names = list(train_data.class_indices.keys())
print("Classes:")
for i, name in enumerate(class_names):
    print(i, ":", name)
images, labels = next(train_data)
plt.figure(figsize=(12, 8))
for i in range(9):
    plt.subplot(3, 3, i + 1)
    plt.imshow(images[i])
    plt.title(class_names[np.argmax(labels[i])])
    plt.axis("off")
plt.tight_layout()
plt.show()
```

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu',
                  input_shape=(128, 128, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(len(class_names), activation='softmax')
])
model.summary()
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
EPOCHS = 15
history = model.fit(
    train_data,
    validation_data=validation_data,
    epochs=EPOCHS
)
plt.figure(figsize=(8, 5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

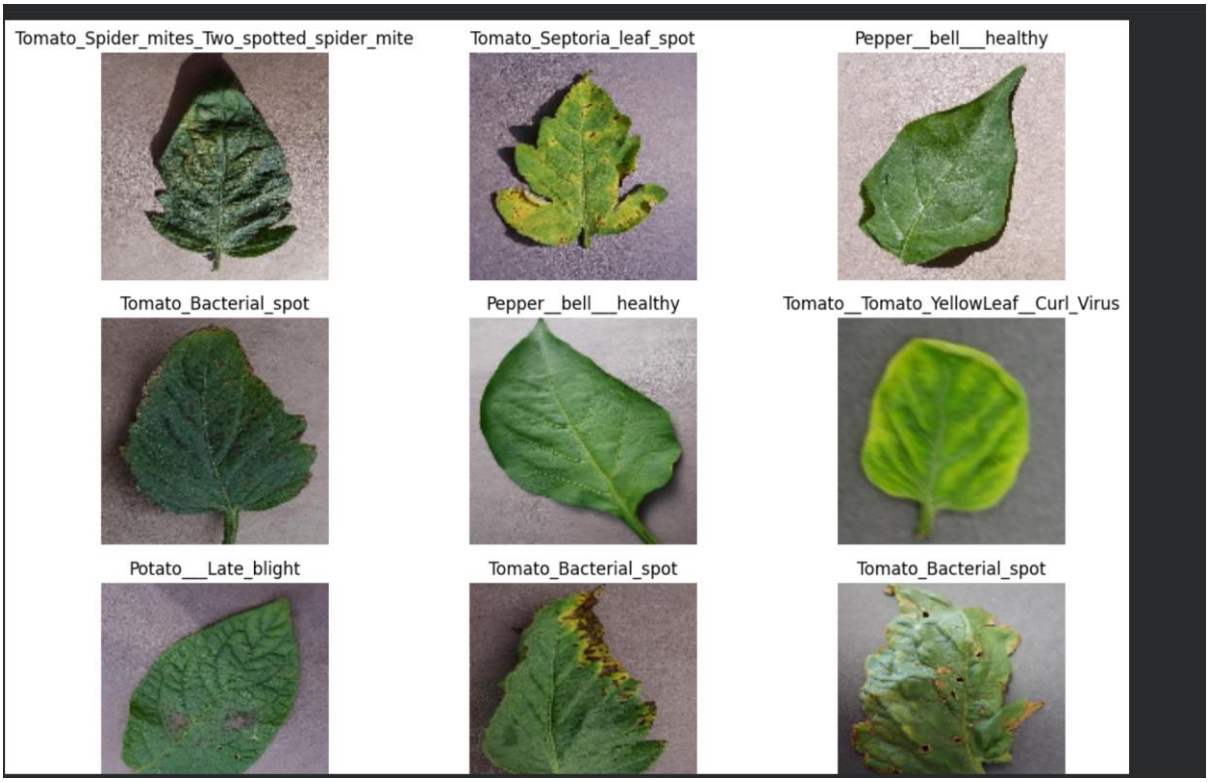
plt.show()
```

```
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
loss, accuracy = model.evaluate(validation_data)
print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)
validation_data.reset()
predictions = model.predict(validation_data)
predicted_classes = np.argmax(predictions, axis=1)
actual_classes = validation_data.classes
print(classification_report(
    actual_classes,
    predicted_classes,
    target_names=class_names
))
cm = confusion_matrix(
    actual_classes,
    predicted_classes
)
plt.figure(figsize=(12, 10))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    xticklabels=class_names,
    yticklabels=class_names
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.show()
from google.colab import files
uploaded = files.upload()
from tensorflow.keras.preprocessing import image
filename = list(uploaded.keys())[0]
img = image.load_img(
    filename,
    target_size=(128, 128)
)
img_array = image.img_to_array(img)
img_array = img_array / 255.0
img_array = np.expand_dims(img_array, axis=0)
prediction = model.predict(img_array)
predicted_index = np.argmax(prediction)
predicted_class = class_names[predicted_index]
confidence = prediction[0][predicted_index] * 100
```

```
plt.imshow(img)
plt.axis("off")
plt.title(
    f"Prediction: {predicted_class}\n"
    f"Confidence: {confidence:.2f}%"
)
plt.show()
print("Predicted Disease:", predicted_class)
print("Confidence:", f"{confidence:.2f}%")
model.save("tomato_disease_cnn.h5")
```

Output




```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 15)	1,935

```
Total params: 3,306,575 (12.61 MB)
Trainable params: 3,306,575 (12.61 MB)
Non-trainable params: 0 (0.00 B)
```

```
EPOCHS = 15

history = model.fit(
    train_data,
    validation_data=validation_data,
    epochs=EPOCHS
)
```

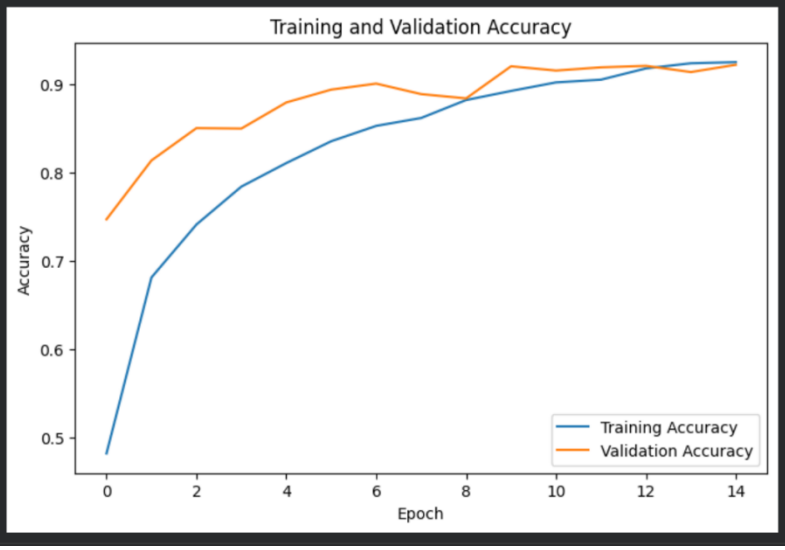
```
Epoch 1/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 38s 59ms/step - accuracy: 0.4816 - loss: 1.6106 - val_accuracy: 0.7470 - val_loss: 0.7944
Epoch 2/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 26s 49ms/step - accuracy: 0.6810 - loss: 0.9601 - val_accuracy: 0.8137 - val_loss: 0.5328
Epoch 3/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 25s 48ms/step - accuracy: 0.7410 - loss: 0.7669 - val_accuracy: 0.8503 - val_loss: 0.4548
Epoch 4/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 25s 48ms/step - accuracy: 0.7840 - loss: 0.6285 - val_accuracy: 0.8498 - val_loss: 0.4429
Epoch 5/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 25s 49ms/step - accuracy: 0.8107 - loss: 0.5504 - val_accuracy: 0.8794 - val_loss: 0.3543
Epoch 6/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 25s 48ms/step - accuracy: 0.8355 - loss: 0.4952 - val_accuracy: 0.8940 - val_loss: 0.3253
Epoch 7/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 24s 46ms/step - accuracy: 0.8529 - loss: 0.4220 - val_accuracy: 0.9008 - val_loss: 0.2961
Epoch 8/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 25s 48ms/step - accuracy: 0.8618 - loss: 0.3969 - val_accuracy: 0.8889 - val_loss: 0.3512
Epoch 9/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 25s 48ms/step - accuracy: 0.8822 - loss: 0.3312 - val_accuracy: 0.8840 - val_loss: 0.3404
Epoch 10/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 25s 48ms/step - accuracy: 0.8925 - loss: 0.3058 - val_accuracy: 0.9204 - val_loss: 0.2443
Epoch 11/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 24s 47ms/step - accuracy: 0.9022 - loss: 0.2768 - val_accuracy: 0.9156 - val_loss: 0.2776
Epoch 12/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 26s 50ms/step - accuracy: 0.9052 - loss: 0.2675 - val_accuracy: 0.9192 - val_loss: 0.2430
Epoch 13/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 25s 48ms/step - accuracy: 0.9180 - loss: 0.2255 - val_accuracy: 0.9209 - val_loss: 0.2834
Epoch 14/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 24s 46ms/step - accuracy: 0.9238 - loss: 0.2141 - val_accuracy: 0.9139 - val_loss: 0.2950
Epoch 15/15
517/517 ━━━━━━━━━━━━━━━━━━━━ 24s 45ms/step - accuracy: 0.9251 - loss: 0.2118 - val_accuracy: 0.9221 - val_loss: 0.2531
```

```
plt.figure(figsize=(8, 5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```

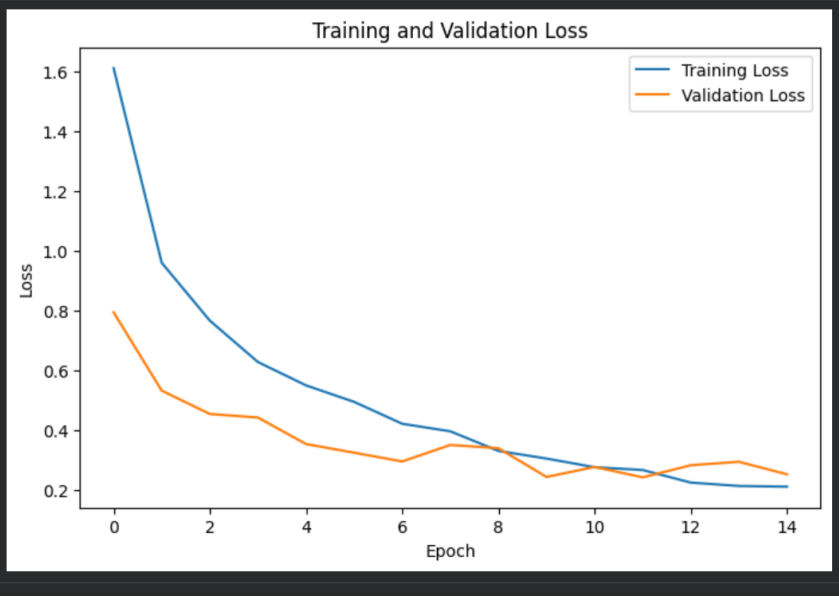


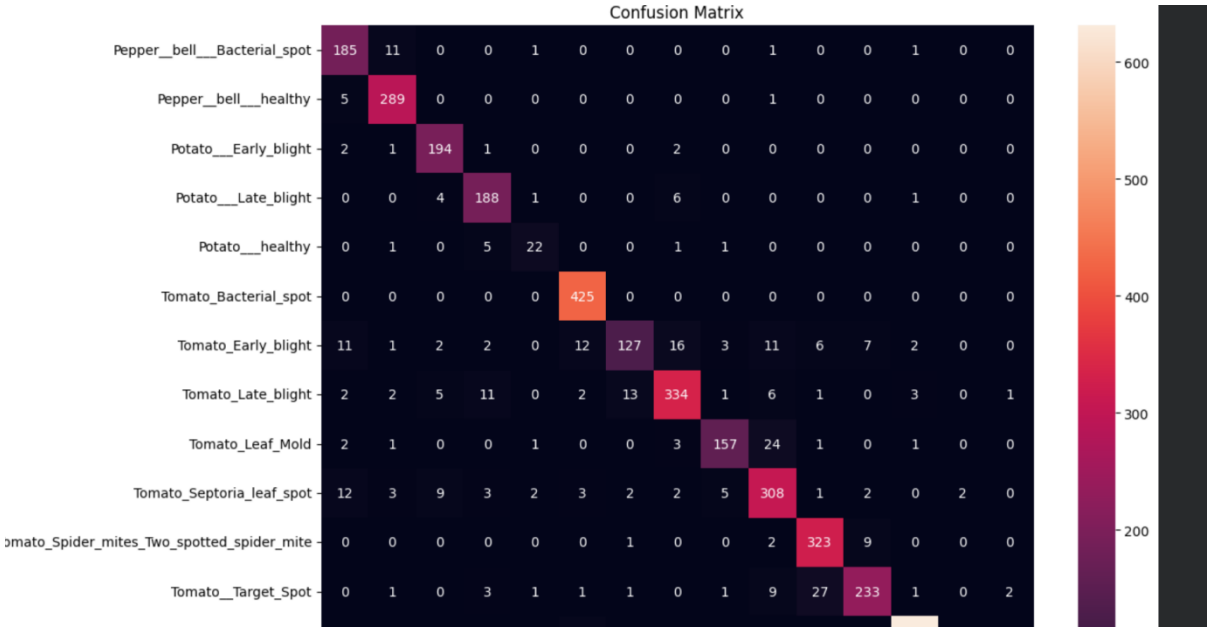
```
plt.figure(figsize=(8, 5))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.show()
```





```
plt.imshow(img)
plt.axis("off")
plt.title(
    f"Prediction: {predicted_class}\n"
    f"Confidence: {confidence:.2f}%"
)
plt.show()

print("Predicted Disease:", predicted_class)
print("Confidence:", f"{confidence:.2f}%")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 675ms/step
```




```
Predicted Disease: Tomato_Bacterial_spot
Confidence: 100.00%
```

```python
import os

for root, dirs, files in os.walk("tomato_dataset"):
    print(root)
    if len(files) > 0:
        print("Number of images:", len(files))
    if root.count(os.sep) > 3:
        dirs[:] = []
```

```
tomato_dataset
tomato_dataset/plantvillage
tomato_dataset/plantvillage/PlantVillage
tomato_dataset/plantvillage/PlantVillage/Pepper__bell___Bacterial_spot
Number of images: 997
tomato_dataset/plantvillage/PlantVillage/Tomato_Septoria_leaf_spot
Number of images: 1771
tomato_dataset/plantvillage/PlantVillage/Tomato_Early_blight
Number of images: 1000
tomato_dataset/plantvillage/PlantVillage/Tomato_healthy
Number of images: 1591
tomato_dataset/plantvillage/PlantVillage/Potato___Early_blight
Number of images: 1000
tomato_dataset/plantvillage/PlantVillage/Tomato__Tomato_YellowLeaf__Curl_Virus
Number of images: 3209
tomato_dataset/plantvillage/PlantVillage/Tomato_Leaf_Mold
Number of images: 952
tomato_dataset/plantvillage/PlantVillage/Potato___Late_blight
Number of images: 1000
tomato_dataset/plantvillage/PlantVillage/Tomato__Target_Spot
Number of images: 1404
tomato_dataset/plantvillage/PlantVillage/Tomato_Spider_mites_Two_spotted_spider_mite
Number of images: 1676
tomato_dataset/plantvillage/PlantVillage/Tomato_Late_blight
Number of images: 1909
tomato_dataset/plantvillage/PlantVillage/Tomato_Bacterial_spot
Number of images: 2127
tomato_dataset/plantvillage/PlantVillage/Pepper__bell___healthy
Number of images: 1478
tomato_dataset/plantvillage/PlantVillage/Tomato__Tomato_mosaic_virus
Number of images: 373
tomato_dataset/plantvillage/PlantVillage/Potato___healthy
Number of images: 152
```